FGE Lens Engine v1 — SPEC TIER

Status: Buildable now. Every claim in this document is falsifiable and testable.
Scope: N+1 through N+4 only. Nothing past this point is in this document.

A companion document, `FGE_Lens_Engine_HYPOTHESIS_TIER.md`, holds N+5–N+10 as explicitly unproven direction — no implementation commitment, no effort estimates, no metrics presented as real until N+4 exists and the data says otherwise.

---

0. What This Is

A control layer that sits above the six FGE departments (Governance Bureau, Material Matrices, Narrative House, Character Division, Marketing Armory, Intelligence Directorate). A **Lens** is a weighted view across department outputs. A **Compound Lens** stacks multiple lenses with a defined blend rule. The engine lets you compose, weight, and auto-select these views from a bird's-eye position — without opening any single department's working project.

This is not a cognition engine, not a perception field, not self-modifying software. It is configuration + routing + weighting over Context Blocks. That's the whole claim.

---

1. Formal Schemas

1.1 Atomic Lens (N+1)

```
{
  "id": "IdentityLens_v1",
  "version": "1.0",
  "included_layers": ["L0", "L4"],
  "excluded_layers": ["L2"],
  "weights": {"L0": 1.0, "L1": 0.0, "L2": 0.0, "L3": 0.0, "L4": 1.0, "L5": 0.0},
  "distortion_profile": {"curvature": 0.0},
  "output_bias": {"verbosity": 0.8}
}
```

- `included_layers` / `excluded_layers` : which OS layers (or departments) this lens

boosts or mutes.

- `weights` : numeric weight per layer/department, 0 (ignore) to 1 (max).
- `distortion_profile` / `output_bias` : optional fine-grained, non-mandatory tuning. Safe to leave default.

For FGE's actual structure, swap L0–L5 for your six departments:

```
{
  "id": "LaunchLens_v1",
  "version": "1.0",
  "included_layers": ["CHARACTER_DIVISION", "MARKETING_ARMORY"],
  "excluded_layers": ["GOVERNANCE_BUREAU"],
  "weights": {
    "GOVERNANCE_BUREAU": 0.0,
    "MATERIAL_MATRICES": 0.2,
    "NARRATIVE_HOUSE": 0.3,
    "CHARACTER_DIVISION": 1.0,
    "MARKETING_ARMORY": 1.0,
    "INTELLIGENCE_DIRECTORATE": 0.1
  }
}
```

1.2 Compound Lens (N+1)

```
{
  "id": "launch_review_lens",
  "version": "1.0",
  "stack": [
    {"id": "LaunchLens_v1"},
    {"id": "IdentityLens_v1"}
  ],
  "blend_mode": "weighted",
  "conflict_resolution": "override"
}
```

Blend modes:

- `weighted` — combine proportionally (see 3.1 algorithm)
- `dominant` — first lens in stack wins outright
- ~~`oscillating`~~ — deferred to Hypothesis tier (requires N+5 parallel execution, not in scope here)

Conflict resolution:

- `override` — top-of-stack wins
- `merge` — average the conflicting weights
- `suppress` — nullify the conflicting pair (neither applies)

1.3 Lens Evolution Rules (N+2)

```
{
  "performance_threshold": 0.8,
  "amplify_on": ["success_rate", "throughput"],
  "suppress_on": ["error_rate", "latency"],
  "mutation_rate": 0.0,
  "reweight_factor": 1.1
}
```

Note: `mutation_rate` is set to 0.0 by default in Spec tier. Random mutation is an N+3 generative concept — explicitly excluded here. N+2 in this document means **rule-based reweighting only**: if a lens underperforms against a defined metric, its weight shifts by a fixed, deterministic factor. No randomness, no spontaneous trait drift. That's the honest boundary of "adaptive" at this tier.

---

2. System Components

Component	Job	Minimal API
Lens State Engine	Stores Lens / CompoundLens definitions, versioned	`GET /lens`, `POST /lens`, `GET /lens/{id}`
Blend Engine	Computes final weight map from a stack	internal function, not exposed
Context Classifier	Maps a task description to a lens stack (N+4)	`POST /classify`
Governance Gate	Validates a lens doesn't violate locked rules before it's saved or applied	`POST /validate`

That's the full component list for Spec tier. No Evolution Engine service, no Meta-Control Compiler, no Agent Router, no Ledger blockchain, no distributed consensus — those are

Hypothesis-tier concepts (N+6 and beyond) and are out of scope here.

---

3. Runtime Algorithms

3.1 N+1 — Weighted Blend

```
def blend_lenses(lens_list):
    layers = ["GOVERNANCE_BUREAU", "MATERIAL_MATRICES", "NARRATIVE_HOUSE",
              "CHARACTER_DIVISION", "MARKETING_ARMORY", "INTELLIGENCE_DIRECTORATE
    total_weight = {L: 0.0 for L in layers}
    for lens in lens_list:
        for L in layers:
            total_weight[L] += lens.weights.get(L, 0.0)
    max_w = max(total_weight.values()) or 1.0
    for L in layers:
        total_weight[L] = min(total_weight[L] / max_w, 1.0)
    return total_weight
```

Test: Combine known lens weights, verify the result sums and clamps correctly (no department weight exceeds 1.0 or drops below 0.0). **Metric:** Numeric accuracy of the blend; latency (should be microseconds — this is arithmetic, not AI).

3.2 N+2 — Deterministic Reweighting

```
def reweight_lens(lens, feedback, rules):
    if feedback.success_rate < rules.performance_threshold:
        for layer in rules.suppress_on:
            if layer in lens.weights:
                lens.weights[layer] *= (1 - (rules.reweight_factor - 1))
    elif feedback.success_rate >= rules.performance_threshold:
        for layer in rules.amplify_on:
            if layer in lens.weights:
                lens.weights[layer] = min(lens.weights[layer] * rules.reweight_fa
    return lens
```

Test: Simulate a feedback scenario (success rate drop), confirm reweighting moves in the predicted direction by the predicted amount — no more, no less. **Metric:** Convergence is deterministic — same input always produces same output. If it doesn't, the implementation is wrong, not "adapting."

3.3 N+3 — Out of Spec Tier

Genuine lens hybridization (genome crossover, spawning new lenses) requires defining what a "successful" lens even means well enough to breed toward it — that measurement framework doesn't exist yet. **Deferred to Hypothesis tier in full.** Nothing in this section is built.

3.4 N+4 — Context-Based Lens Selection

```
def select_lens_for_context(context_text, rules):
    if "launch" in context_text.lower():
        return ["LaunchLens_v1", "IdentityLens_v1"]
    if "audit" in context_text.lower():
        return ["GovernanceLens_v1"]
    return ["DefaultLens_v1"]
```

Start rule-based. Do not reach for an ML classifier until rule-based selection has been used and has visibly failed — premature ML here is solving a problem you don't have yet.

Test: Provide sample context strings, verify the correct lens stack is selected. **Metric:** Selection accuracy against a hand-labeled test set; classification latency under 50ms (trivial for rule-based lookup).

---

4. Governance Rule (carried forward regardless of build progress)

> **Identity Lens cannot be altered by Strategy.** The core system identity (Governance Bureau's constitutional outputs, canon locks) is sacrosanct. Other lenses may observe it, weight around it, or exclude it from a blend — but no lens, compound lens, or future evolution mechanism may write to or override Governance Bureau's outputs.

This rule should be enforced in the Governance Gate component (`POST /validate`) before any lens definition is saved, from day one — even in the most minimal prototype.

---

5. Minimal Viable Implementation

```
fge-lens/
├─ backend/
│  ├─ main.py            # FastAPI app
│  ├─ models.py          # Pydantic schemas: Lens, CompoundLens
```

```
│  ├─ state_engine.py    # CRUD for lens storage
│  ├─ blend.py           # blend_lenses(), reweight_lens()
│  ├─ classify.py        # select_lens_for_context()
│  └─ governance.py      # validate(): enforces Identity Lens rule
└─ README.md
```

```
# models.py
from pydantic import BaseModel
from typing import List, Dict

class Lens(BaseModel):
    id: str
    version: str
    included_layers: List[str]
    excluded_layers: List[str] = []
    weights: Dict[str, float]
    distortion_profile: Dict[str, float] = {}
    output_bias: Dict[str, float] = {}
```

```
# main.py
from fastapi import FastAPI
from models import Lens
from state_engine import LensStore
from governance import validate

app = FastAPI()
store = LensStore()

@app.post("/lens", response_model=Lens)
def create_lens(lens: Lens):
    validate(lens)  # raises if it violates the Identity Lens rule
    store.save(lens)
    return lens

@app.get("/lens")
def list_lenses():
    return store.list_all()

@app.post("/classify")
def classify(context_text: str):
    from classify import select_lens_for_context
    return select_lens_for_context(context_text, rules=None)
```

No Three.js dashboard, no holographic UI, no WebSocket scene graph in this tier. A REST client or simple HTML form listing lenses is sufficient to prove the mechanism

works. Visualization is a later layer built on top of a working engine — not a precondition for one.

---

6. Definition of Done for Spec Tier

You'll know N+1–N+4 actually exists, not just on paper, when:

1. You can `POST` a lens with department weights and `GET` it back unchanged.
2. Two lenses stacked with `blend_mode: weighted` produce a single, correctly-normalized weight map.
3. A lens that violates the Identity Lens governance rule is rejected by `/lens`, not silently accepted.
4. Given a plain-text context string, `/classify` returns a sensible lens stack at least 80% of the time against a test set you write yourself (even 10 examples is enough to start).
5. Nothing in this system requires you to describe what it does using the words "consciousness," "field," "emergent," or "singularity." If you find yourself reaching for that vocabulary to explain something in this tier, it has drifted into Hypothesis territory and should be flagged, not built.